

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 10

Add Your Own System Call: trace

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: The number and the user side 2
2.2 Part 2: The kernel side 4
2.3 Part 3: Play with the mask 6
2.3.1 Observe Answers (Part 3) 8

3 CONCLUSIONS **9**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The objective of this laboratory work is to change the kernel of the xv6 operating system for the first time, by adding a new system call named `trace`. The call makes a process print every system call it makes, in the same way the `strace` tool did in Laboratory Work No. 9. The work follows one number, 23, from a user program down to a function in the kernel.

1.2 Theoretical Background

In xv6 for RISC-V, a user program asks the kernel for a service through a chain of steps, and adding a system call means adding one piece to each step of this chain:

- **System call number.** Every system call has a number, defined in the kernel file `syscall.h`. The number is the only thing that identifies the call between user space and the kernel.
- **User-side stub.** The script `user/usys.pl` generates, at build time, a small assembly function for each call. For `trace` it contains three instructions: `li a7, SYS_trace` (put the call number in register `a7`), `ecall` (switch to the kernel) and `ret`.
- **Trap into the kernel.** The `ecall` instruction moves the processor into supervisor mode and jumps to the trap handler, which saves the registers of the process in its `trapframe` and calls the function `syscall()` from `kernel/syscall.c`.
- **Dispatch.** The function `syscall()` reads the number from `a7`, uses it as an index in the table `syscalls[]` of function pointers, calls the handler, and stores the returned value in `a0`, which becomes the return value of the stub in user space. The arguments of the call are read by the handler from registers `a0`–`a5` with functions such as `argint`.
- **Trace mask.** Each process keeps an integer mask in its `struct proc`. The mask is read in binary: bit N set means "print system call number N". For example, $32 = 2^5$ and `SYS_read` is 5, so the mask 32 traces only `read`. A child created by `fork` copies the mask of its parent, and `exec` keeps it, because `exec` replaces the program but not the process.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: The number and the user side

The first part gives the call a number and teaches the user side how to ask for it. Five small changes were made.

1.1 `kernel/syscall.h`: the number. One line was added at the end of the file:

```
#define SYS_sync    22
#define SYS_trace   23      // new
```

1.2 `user/user.h`: the declaration. The prototype of the new call was added after the other system calls:

```
int sync(void);
int trace(int);            // new
```

1.3 `user/usys.pl`: the stub that runs `ecall`. One line was added at the end. At build time this script writes `user/usys.S`, where `trace` becomes the three instructions `li a7, SYS_trace`, `ecall`, `ret`.

```
entry("sync");
entry("trace");           # new
```

1.4 `user/trace.c`: the program. The new program sets the mask on itself and then becomes the requested command with `exec`. The mask stays, because `exec` keeps the process.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

1.5 `Makefile`: `trace` on the xv6 disk. In the `UPROGS` list, after `$U/_zombie`, the following line was added (it starts with a TAB):

```
        $U/_zombie\
        $U/_trace\
```

Check 1: the number reaches the kernel. After `make qemu`, the command `trace 32 grep hello README` was run. The kernel has no function for number 23 yet, so it must complain, and Figure 2.1 shows that it does: the message `unknown sys call 23` was printed by the kernel, and then `trace` printed `trace: failed`. This is the expected result, because it proves that the program executed `ecall` with `a7 = 23` and that the kernel received the number.

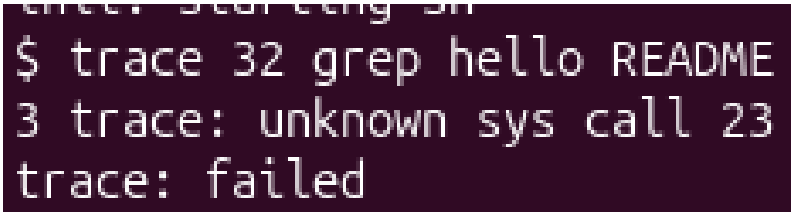


Figure 2.1: Check 1: the kernel receives call number 23 but does not know it yet

2.2 Part 2: The kernel side

The second part gives the kernel a function for number 23 and makes it print the calls.

2.1 `kernel/proc.h`**: a place for the mask.** Each process remembers its own mask, so a new field was added to `struct proc`, after the `name` field:

```
  char name[16];               // Process name (debugging)
  int trace_mask;              // new: which calls to print
```

2.2 `kernel/sysproc.c`**: the handler.** The function `sys_trace` was added at the end of the file. It reads argument 0 (register `a0`) and stores it in the current process:

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

2.3 `kernel/syscall.c`**: the table.** Two lines tell the file that the function exists and give it place 23 in the table of handlers:

```
extern uint64 sys_sync(void);
extern uint64 sys_trace(void);   // new
...
  [SYS_sync]    = sys_sync,
  [SYS_trace]   = sys_trace,       // new
```

2.4 `kernel/syscall.c`**: the names and the print.** The kernel only knows numbers, so a table of names was added just above the function `syscall()`:

```
static char *syscallnames[] = {
  [SYS_fork] "fork",   [SYS_exit] "exit",     [SYS_wait] "wait",
  [SYS_pipe] "pipe",   [SYS_read] "read",     [SYS_kill] "kill",
  [SYS_exec] "exec",   [SYS_fstat] "fstat",   [SYS_chdir] "chdir",
  [SYS_dup] "dup",     [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
  [SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
  [SYS_write] "write", [SYS_mknod] "mknod",   [SYS_unlink] "unlink"
    ,
  [SYS_link] "link",   [SYS_mkdir] "mkdir",   [SYS_close] "close",
  [SYS_sync] "sync",   [SYS_trace] "trace",
};
```

Then, inside `syscall()`, two lines were added right after the line that runs the call. They are placed after the call, so the return value is already in `a0` when it is printed. The kernel print function in this version of xv6 is `printk`.

```
    p->trapframe->a0 = syscalls[num]();
    if ((p->trace_mask >> num) & 1)                    // new
      printk("%d: syscall %s -> %ld\n",                // new
             p->pid, syscallnames[num], p->trapframe->a0);
```

2.5 `kernel/proc.c`**: children inherit the mask.** In `fork()`, right after the line that copies the name, the mask is copied from the parent to the child:

```
  safestrcpy(np->name, p->name, sizeof(p->name));
  np->trace_mask = p->trace_mask;       // new
```

Check 2: trace works. After rebuilding the system with `make qemu`, the command `trace 32 grep hello README` prints, as shown in Figure 2.2, one line for every `read` made by `grep`. The numbers are $1023 + 965 + 453 = 2441$, which is the size of the file `README` in bytes, read in three pieces, and the final 0 means the end of the file. The number at the start of each line is the pid of the traced process.



Figure 2.2: Check 2: `trace 32 grep hello README` prints every `read` call

2.3 Part 3: Play with the mask

The mask is a number read in binary: bit N on means "print call number N". The system calls used in the experiments have the numbers `SYS_fork` = 1, `SYS_read` = 5, `SYS_exec` = 7, `SYS_open` = 15, `SYS_write` = 16 and `SYS_close` = 21. Table 1 summarizes the masks used.

Mask	Bits set	Traced calls
32	2^5	`read`
2	2^1	`fork`
65536	2^{16}	`write`
2129920	$2^{15} + 2^{21}$	`open` and `close`
2147483647	$2^{31} - 1$ (bits 0–30)	all calls

Table 1: Masks used in the experiments

3.1 All the calls. With the mask 2147483647, all bits are on, so every system call of the process is printed (Figure 2.3). The first line is `trace` itself, returning 0. Then `exec` returns 3, which is the number of arguments of the new program (`grep hello README`), `open` returns the descriptor 3, the three `read` calls return the same sizes as before, and `close` returns 0.

```
$ trace 2147483647 grep hello README
4: syscall trace -> 0
4: syscall exec -> 3
4: syscall open -> 3
4: syscall read -> 1023
4: syscall read -> 965
4: syscall read -> 453
4: syscall read -> 0
4: syscall close -> 0
```

Figure 2.3: `trace 2147483647 grep hello README`: all system calls of `grep`

3.2 Only `fork`. The command `trace 2 usertests forkforkfork` traces only `fork` (bit 1) in a test that creates processes until the process table is full. Figure 2.4 shows the end of the output: the lines come from several different pids, and the last `fork` calls return −1, which means that no more processes can be created. The test finishes with `OK` and `ALL TESTS PASSED`.

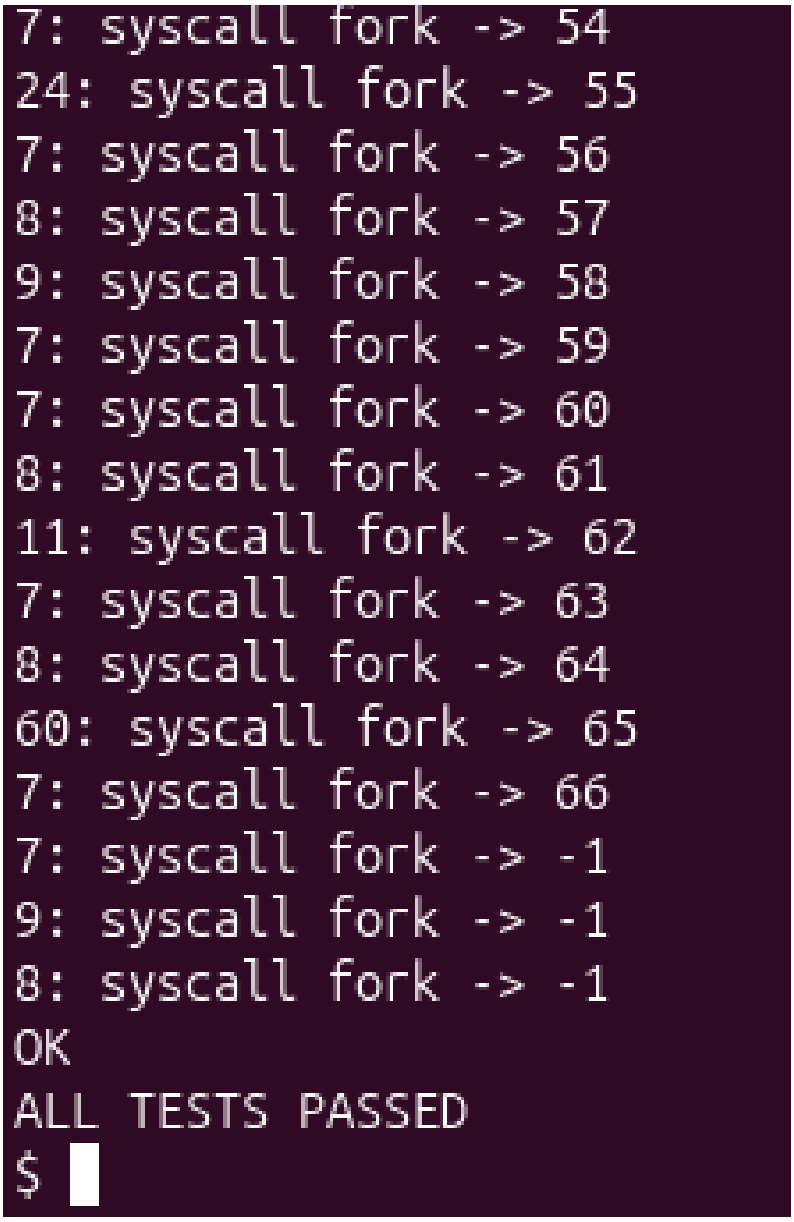


Figure 2.4: `trace 2 usertests forkforkfork`: only `fork` is traced, in many processes

3.3 Only `write`. The command `trace 65536 echo hi` traces only `write` (bit 16). In Figure 2.5 the trace line appears between `hi` and the new line.

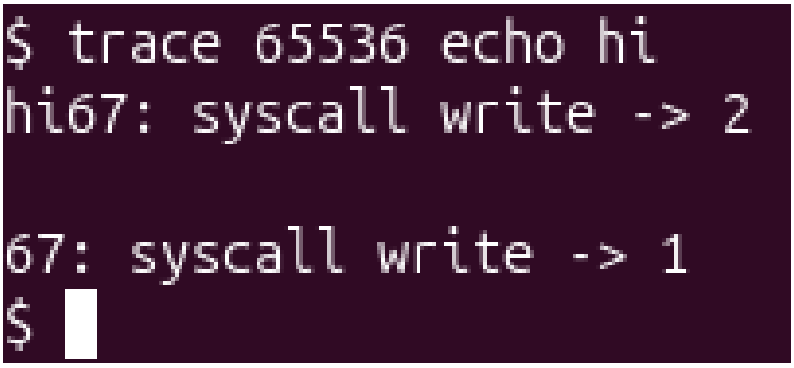


Figure 2.5: `trace 65536 echo hi`: the trace line is printed between `hi` and the new line

3.4 Only `open` and `close`. The mask for both calls is $2^{15} + 2^{21} = 32768 + 2097152 = 2129920$. Figure 2.6 shows that only `open` and `close` are printed, and nothing else.

```
$ trace 2129920 grep hello README
68: syscall open -> 3
68: syscall close -> 0
$
```

Figure 2.6: `trace 2129920 grep hello README`: only `open` and `close`

2.3.1 Observe Answers (Part 3)

1. **Why only `read` in `trace 32 grep hello README`?** The mask 32 is 2^5, so only bit 5 is on, and 5 is the number of `SYS_read`. The numbers of `open` (15) and `close` (21) are not set in the mask, so the test `(p->trace_mask » num) & 1` is false for them and `syscall()` does not print them.

2. **Which line makes several pids print in `forkforkfork`?** The line `np->trace_mask = p->trace_mask;` added to `fork()` in `kernel/proc.c` (Part 2.5). It copies the mask of the parent to every child, so the children, and their own children, print their `fork` calls too, each with its own pid.

3. **Why is the trace line between "hi" and the new line in `trace 65536 echo hi`?** `echo` calls `write` twice: first for the text `hi` (2 bytes), then for the new line character (1 byte). The kernel prints the trace line after each call finishes. Therefore the output is: `hi`, then the line `write -> 2` of the first call, then the new line written by the second call, then the line `write -> 1` of the second call.

4. **Which mask traces both `open` (15) and `close` (21), and nothing else?** $2^{15} + 2^{21} = 32768 + 2097152 = 2129920$.

3 CONCLUSIONS

In this laboratory work I added a new system call, `trace`, to the xv6 kernel. I followed the number 23 along the whole path: the definition in `syscall.h`, the declaration in `user.h`, the stub generated by `usys.pl` that runs `ecall`, the entry in the table of handlers, and the handler `sys_trace` that stores the mask in the `struct proc` of the process. Check 1, with the message `unknown sys call 23`, showed that the user side works before the kernel side exists.

Furthermore, the experiments with the mask showed that a system call number is a bit position, that the print is done by `syscall()` after the call, so the return value is available, and that the mask must be copied in `fork()` to be inherited by the children. In Laboratory Work No. 9 the Linux tool `strace` showed the system calls from the outside; here the same information is produced by the kernel itself, which makes the boundary between user space and the kernel clearer.